DESIGNING AN AGENT HARNESS FOR MACHINE LEARNING DEVELOPMENT

Weiwei Wu
Stanford University
wwwu@stanford.edu

1 RESEARCH QUESTIONS OR PROBLEM SPECIFICATION

ML engineers repeatedly inspect results, choose model changes, and run experiments. We are building one agentic system to carry out this loop across MLPs, CNNs, and transformers. Given a dataset, starting model, and compute budget, it proposes changes and tests them. Our current question is: **Do detailed error reports and a separate reviewer help an agent produce better models within the same compute budget?** Following the proposal feedback, we compare four fixed harness designs to study how agents use evidence and coordinate decisions. Improving the workflow itself remains a later step.

2 ENVIRONMENT & DATA

Tasks. One planner and runner use task-specific data loaders, builders, and graders (Table 1). Our completed evaluation uses UCI Covertype (Blackard, 1998). Its 581,012 labeled land cells have 54 terrain and soil features and seven forest cover classes, derived from US Forest Service and US Geological Survey records. Covertype replaced a Spambase pilot that left little room for improvement.

Table 1: Three model families. Only the Covertype study has completed the current evaluation.

Model	Task and dataset	Starting model	Score; status
MLP	Forest cover classification; Covertype	Two hidden layers: 128, 64 units; from scratch	Accuracy; completed
CNN	100-class image classification; CIFAR-100	CIFAR ResNet-18; from scratch	Top-1 accuracy; runs pending
Transformer	Sentiment: SST-2; paraphrases: MRPC	Pretrained BERT-Tiny with a fresh classifier	Accuracy; MRPC F1; runs pending

Evaluation data. We construct a stratified 60/20/20 train/validation/test split of Covertype with split seed 20261104. Preprocessing statistics are fitted on training data only. The downloaded archive hash and split indices are recorded. Agents see training and validation results. A separate evaluator scores the selected model on test labels hidden from the agents. The planned image and text tasks use CIFAR-100 and GLUE (Krizhevsky, 2009; Wang et al., 2019); their evaluation runs are pending.

Resources and allowed changes. The Covertype baseline is a $54 \to 128 \to 64 \to 7$ MLP trained for five epochs with AdamW, learning rate 0.001, batch size 128, cosine decay, and dropout 0.1. Agents can change layer sizes, training length, optimizer settings, batch size, schedules, regularization, normalization, and supported preprocessing. Numeric settings are free within constraints, including a 500,000-parameter cap. Each run allows at most 20 attempts and 80 API calls. We evaluated 168-second and 600-second budgets with three matched seeds per design, one NVIDIA L4 per run, and the same `openai/gpt-6-luna` API model.

Success criteria. We measure best validation accuracy against cumulative compute, then test the selected model. We also record attempts, model size, timeouts, API cost, and agent decisions.

Charged compute includes the baseline, worker execution, and diagnostics; API waiting and process startup are recorded separately. Thus equal compute does not mean equal total cost. All scores are calculated by code, not by an LLM judge.

3 METHODS

The harness. As in Homework 1, the harness controls context, tool execution, and stopping. DSPy handles structured model calls (Khattab et al., 2023); Pydantic checks the returned fields; LangGraph connects the stages and checkpoints workflow state (LangChain, n.d.). Trusted Python code builds, trains, and scores each proposed model.

1. **Plan.** Given the task, previous results, and remaining budget, the planner returns an observation, hypothesis, predicted effect, experiment, full model recipe, and supporting trial IDs.
2. **Validate and execute.** Python checks the recipe, parameter count, and budget, then calls the PyTorch worker. Candidates use implemented model builders, with no arbitrary generated-code execution. Matching architectures reuse saved initial weights; changed architectures receive fresh seeded initialization. Candidates never inherit earlier trained weights.
3. **Measure and review.** Every design receives aggregate scores and learning-curve summaries. B and D also receive automatically computed error reports. C and D call a separate reviewer, which states whether the result supports the prediction and recommends a next step. The planner chooses the next experiment; review is advisory.
4. **Record and repeat.** The harness saves results, updates the best model and budget, and repeats. Overlong trials are stopped and charged. The same trained baseline is shared across designs, with its cost charged to each.

Table 2: The two controlled harness choices. Scores, curve summaries, planner instructions, available model changes, and compute limits are shared across A–D.

Design	Detailed error reports	Separate reviewer
A: planner	No	No
B: planner + reports	Yes	No
C: planner + reviewer	No	Yes
D: planner + reports + reviewer	Yes	Yes

Evidence and access. Error reports are computed from validation predictions: classes with low recall, frequent class confusions, confident mistakes, and feature differences between correct and incorrect examples. A and C cannot access these reports, including through the reviewer. B and D receive them automatically, with diagnostic time charged to the budget. B–A and D–C compare report access; C–A and D–B compare review.

Context and memory. Each call includes fixed task constraints, remaining resources, a short record of every attempt, and detailed baseline, best, and recent results. Context details are trimmed to fit; full prompts, outputs, plans, reviews, scores, costs, and failures stay on disk. Validation logits and example IDs are retained at each epoch for later error analysis. We can therefore trace decisions back to their evidence.

4 SUMMARY OF PROGRESS

Implemented and demonstrated. The runner, A–D access rules, model adapters, records, and plotting pipeline are implemented. The two Covertype studies contain 24 A–D workflow runs plus six random-search reference runs. A real-API setup chat saves a reviewed configuration; the demo then shows a completed optimization run. The setup screen does not launch training.

Table 3: Covertype accuracy (%), mean ± sample standard deviation across three seeds on one fixed split. The starting model scores 81.87 ± 0.06 on validation.

Design	Validation, 168 s	Validation, 600 s	Test, 600 s
A	$\mathbf{90.12} \pm 0.70$	93.76 ± 0.94	93.83 ± 1.01
B	86.87 ± 0.25	92.64 ± 2.44	92.68 ± 2.35
C	89.31 ± 2.11	93.41 ± 0.37	93.45 ± 0.36
D	88.91 ± 2.27	$\mathbf{94.13} \pm 0.09$	$\mathbf{94.11} \pm 0.06$

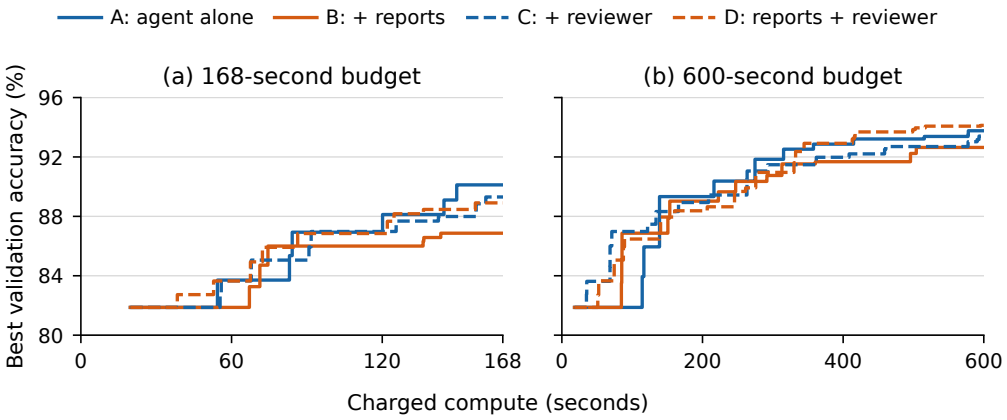


Figure 1: Best validation accuracy versus charged compute, averaged over three seeds. The panels are separate studies with different total budgets. The curves retain the best completed candidate, so their upward steps do not imply that every experiment improved the model.

Preliminary finding. All four designs improve substantially over the five-epoch baseline. Their ranking depends on the budget: A has the highest mean score at 168 seconds, while D leads at 600 seconds (Table 3). The larger budget was chosen after examining the first study. D leads A by 0.36 validation points and 0.28 test points at 600 seconds. Three seeds on one dataset do not establish a general winner.

What the agents actually did. In D's seed-2 run, longer training raised validation accuracy from 81.85% to 87.30%. The reviewer then recommended widening the hidden layers from [128,64] to [256,128] while keeping 25 epochs; the next experiment reached 90.40%. Further width and duration changes reached 94.18% validation and 94.16% test accuracy with 454,279 parameters. The reasoning mainly follows learning curves and model capacity; this example does not establish that detailed reports caused the gains.

Failure modes and obstacles. After that run's best result, the reviewer recommended stopping with only 4.3 seconds left. The planner nevertheless proposed a short learning-rate probe, which hit the budget limit. In another run, B proposed 80 epochs and exceeded the 267-second per-attempt limit. Both expose weaknesses in budget planning. The random-search reference was also weakened by an interaction between class weighting and label smoothing; we therefore do not use its large gap as evidence that agents outperform conventional search.

Next steps. (1) Run the same A–D comparison on CIFAR-100 and the text tasks after checking feasible budgets and locking the text test-label protocol. (2) Compare how each design uses error evidence, follows reviewer advice, allocates training time, and repeats or abandons ideas; pair this analysis with additional seeds. (3) Evaluate a targeted change to budget planning or reviewer coordination, chosen from these failures, against the frozen current harness. The code can propose revised planner instructions, but recursive workflow improvement and its transfer across tasks remain untested.

5 REFERENCES

Jock Blackard. Covertype. UCI Machine Learning Repository, 1998. URL https://archive.ics.uci.edu/dataset/31/covertype. Dataset.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. arXiv preprint arXiv:2310.03714, 2023. URL https://arxiv.org/abs/2310.03714.

Alex Krizhevsky. Learning multiple layers of features from tiny images. University of Toronto, 2009. URL https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf. Technical report.

LangChain. LangGraph overview. Software documentation, n.d. URL https://docs.langchain.com/oss/python/langgraph/overview. Accessed November 6, 2026.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=rJ4km2R5t7.

6 AI-USE DISCLOSURE STATEMENT

ChatGPT/Codex and Claude assisted with project design, implementation, and debugging. Claude assisted with the experiment workflow and earlier result analysis; Codex assisted with the demo interface, checking saved results, and drafting this report. The author selected the direction and constraints and guided the work through iterative discussion. API language-model calls made by the experimental agents are part of the system being evaluated; all reported task scores come from executed training and evaluation code.

CODE AND REPRODUCTION

https://github.com/weiweiwu007/cs329z-ml-agent

The project README describes dependency installation, server-side API-key configuration, running the setup chat and GPU workflow, and regenerating plots from saved results. The repository is private; teaching-team access will be arranged by the author.